交叉注意力的计算原理[1]

假设我们已有一个简化的傅里叶特征库:

$$H(x) \in \mathbb{R}^{n\times d} \quad \text{where} \quad n = \frac{M}{d}$$

$$H_{i,j}(x) = \sin((i-1)d + j)x, \quad i = 1, 2, ..., n, \quad j = 1, 2, ..., d$$

使用坐标 x 生成维度适合的 $Q_0(x)$ 后,Cross-Attention 的计算公式为:

$$\mathrm{CA}(Q_0(x), H(x)) = \mathrm{softmax}\left(\frac{Q_0 W_q W_k^\top H^\top}{\sqrt{d}}\right) H W_v$$

把softmax记为$s = (s_1, s_2, ..., s_n)) \in \mathbb{R}^n$,而$W_v$可以看成下一层的权重矩阵从而省略，从而得到:

$$\mathrm{CA}(Q_0(x), H(x)) = (s_1, s_2, ..., s_n) \cdot H$$

$$
\begin{array}{lcl}
s_1 & \times & [\sin(x), \sin(2x), ..., \sin(dx)] \\
s_2 & \times & [\sin((d+1)x), ..., \sin(2dx)] \\
\vdots & & \vdots \\
s_n & \times & [\sin((n-1)d+1)x), ..., \sin(ndx)]
\end{array}
\longrightarrow \mathrm{CA}(x)
$$

但这也隐含了同一 token 内特征同样重要的假设($\sin(x)$到$\sin(dx)$的系数都是s_1).

并且输出时将不同尺度混在了一起，例如$\sin(x)$输出时不和$\sin(2x)$一组，反而和$\sin((d+1)x)$一组，这打乱了输出结构.

我们期待提出一种方法，赋予每个 token 内部不同特征不同的权重，并且输出具有可解释的结构信息.

我们提出一种名为 KAFF(Kernelized Attention for Fourier Features)的方法，大致思路如下：

- 将物理空间的多尺度 RFF 按「逐个分量频率递增，d整除最后一个分量的频率数」规则划分为 token，每个 token 对应一段连续物理频带
- 每个 token 用输入自适应的一维高斯核生成 token 内单个频率的细粒度权重，token 内聚合得到对应频带的输出
- 解决 RFF-CA 的两个问题：token 内权重共享，输出跨尺度混合

具体如下:

$$\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}, \quad \text{FFN}_1(\boldsymbol{x}) = Q \in \mathbb{R}^h, \quad W_\mu \in \mathbb{R}^{n\times h},$$

$$b_\mu \in \mathbb{R}^h, \quad W_\sigma \in \mathbb{R}^{n\times h}, \quad b_\sigma \in \mathbb{R}^h, \quad \Omega \in \mathbb{R}^{n\times d}$$

下面为了简化，假设 $d_{\text{in}} = 1$,从而频率 ω 都是一维的.

$\Omega = \begin{pmatrix} \omega_1 & \dots & \omega_d \\ \vdots & \dots & \vdots \\ \dots & \dots & \omega_{nd} \end{pmatrix}$,逐行按照频率从低到高排列.

Φ和Ω同形状,$\Phi_{i,j} = \frac{2\pi}{(i-1)d+j}$.

$$H[i,j] = \cos(\Omega[i,j]^T x + \Phi[i,j]), H \in \mathbb{R}^{n\times d}.$$

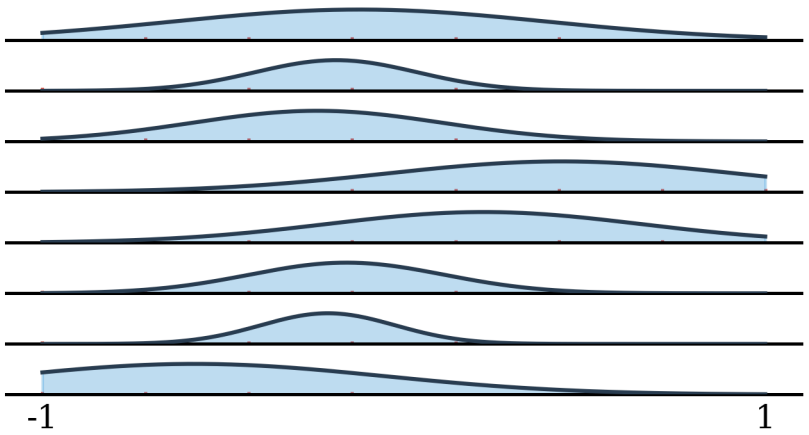
- $G = \Omega$

然后对G按行(在每个token内)归一化到$[-1,1]$, 我们使用G来模拟同一token内频率间的距离.

- $\tanh(W_\mu Q + b_\mu) = \mu$
- $W_\sigma Q + b_\sigma = \sigma$

以高斯核为例：

- $$S[i,j] = \exp\left(-\frac{(G[i,j]-\mu[i])^2}{2\sigma[i]^2}\right)$$
- $A = \mathrm{sum}(S \odot H, \mathrm{dim} = 1) \in \mathbb{R}^n$ （0 代表行，1 代表列）
- $\mathrm{Output} = \mathrm{FFN}_2(A)$



下面考虑高维的情况:

取傅里叶特征频率为网格点(未必是均匀网格),
$\left\{\boldsymbol{\omega}_i = \left(\omega_1, ..., \omega_{d_{\text{in}}}\right) | i = 1, ..., n_1 \times n_2 \times ... \times n_d\right\}$,取$d$整除$n_d$，在划分 token 时使每个 token 内的频率除了最后一维的频率不同以外，其他维度的频率都相同，得到: $\Omega = \begin{pmatrix} \boldsymbol{\omega}_1 & ... & \boldsymbol{\omega}_d \\ \vdots & ... & \vdots \\ ... & ... & \boldsymbol{\omega}_{nd} \end{pmatrix} \in \mathbb{R}^{n \times d}$.

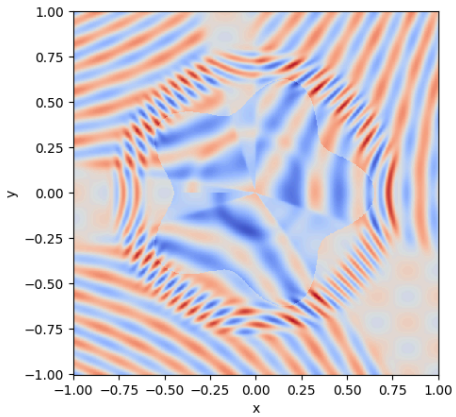
将上文的$G = \Omega$修改为$G = \begin{pmatrix} \boldsymbol{\omega}_1[-1] & ... & \boldsymbol{\omega}_d[-1] \\ \vdots & ... & \vdots \\ ... & ... & \boldsymbol{\omega}_{nd}[-1] \end{pmatrix}$,同一 token 内前面的频率相同,各个特征间的距离只和最后一个维度有关.

得到注意力输出结果后，再通过一个网络得到最终输出$\text{FFN}_2(A)$.

使用残差神经网络拟合如下二维函数，除去输出和输入的线性变换，共使用 4 个残差块.分别设置 4 个残差块中有1，2，3，4个交叉注意力块，其余为普通残差块，普通残差块放置在注意力残差块的后面.

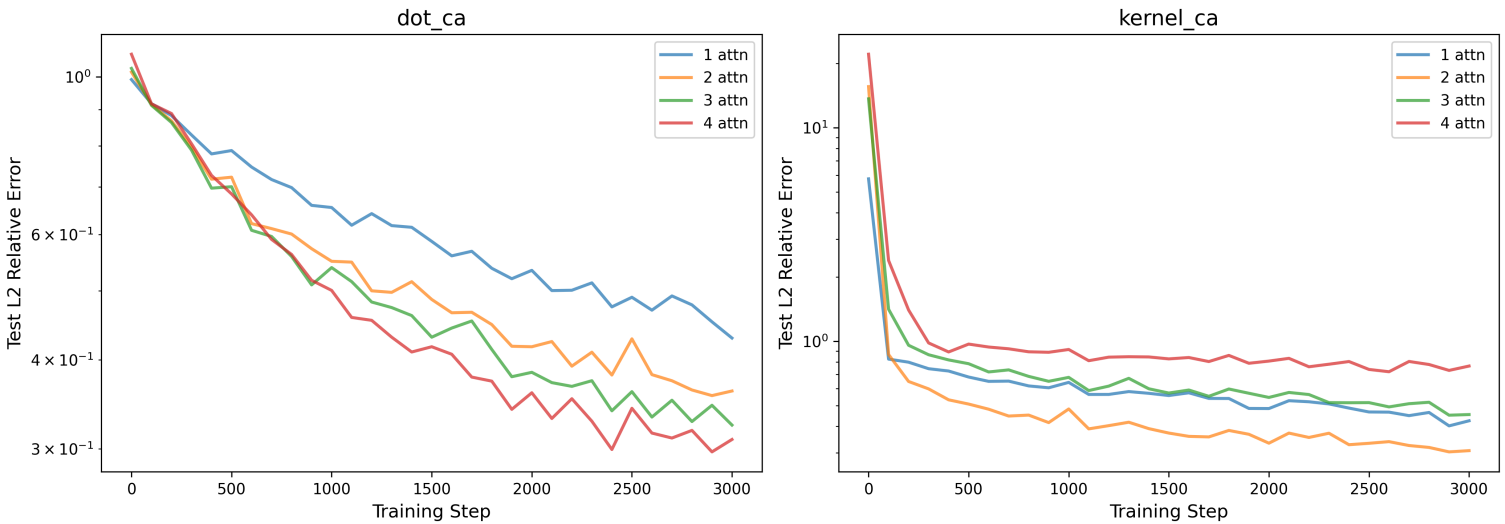
每个残差块的神经网络结构为：

[输入 →(pre_norm → CA → 残差连接)→pre_norm → 线性 1(32→64) → SiLU 激活 → 线性 2(64→32) → 残差连接 → 输出]



使用 Adam 优化器，学习率为 1e-3，取 65536 个训练数据，单个 epoch 使用的 mini batch size 为 8192，dot_ca 方法内使用 4 头注意力,使用均方误差损失函数.

使用核化注意力的方法要比点积注意力的方法训练更慢，注意力层越多训练越慢.



注意力块数	dot_ca 最终误差	kernel_ca 最终误差
1	0.465	0.357
2	0.391	0.302
3	0.317	0.333
4	0.289	0.390

为什么注意力块数的增加没能带来 KAFF 误差的持续下降？

交叉注意力的缺点

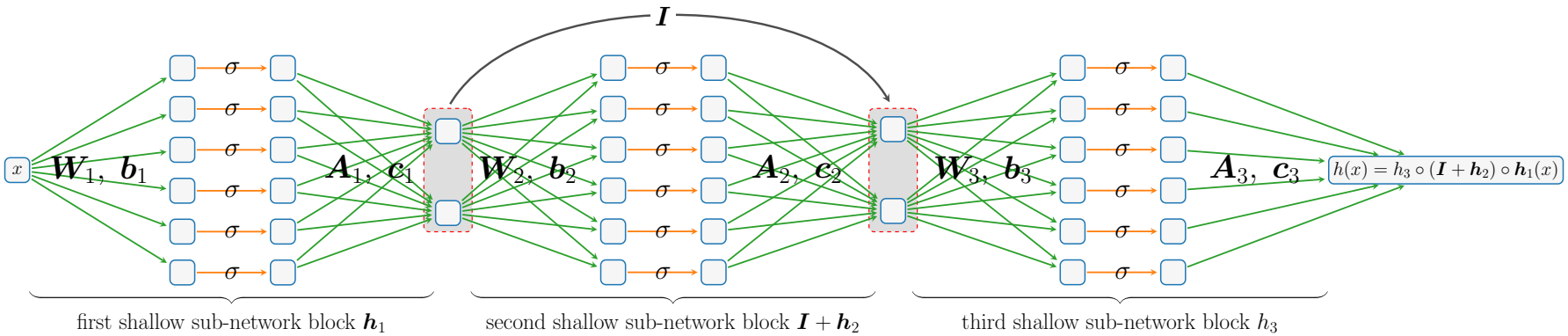
- 1 计算复杂度，显存占用极高.

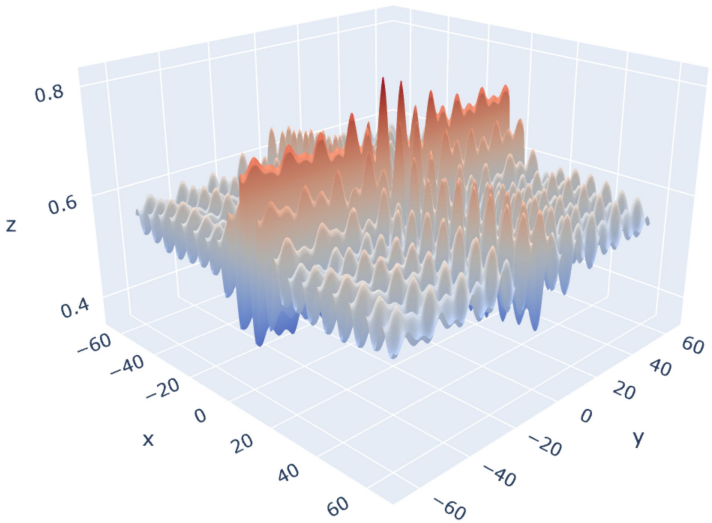
注意力方法本身计算复杂度就较高，在 PINNs 中还需要做高阶自动微分，傅里叶特征库H的每个分量都是由 x 带入非线性函数 cos 生成，不得不保持复杂的计算图.

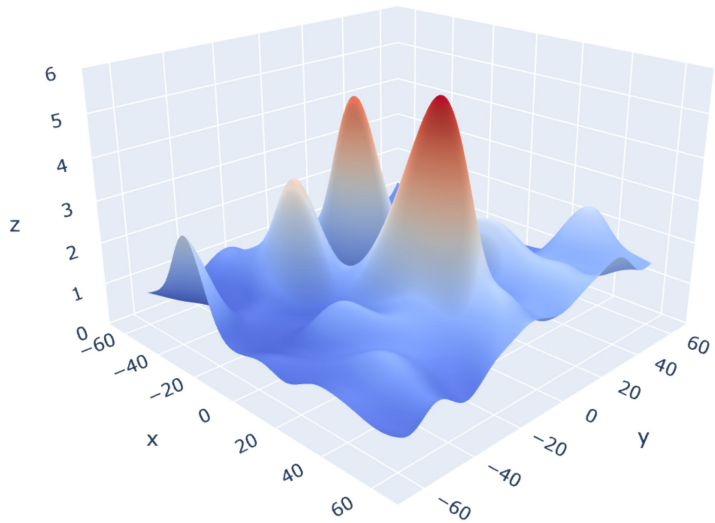
- 2 训练较不稳定

KAFF 方法引入了较强的非线性，这和通常的网络设计中追求可训练参数线性影响输出的原则[2] 相悖，可能导致损失景观过度复杂.

交叉注意力的缺点


$\boldsymbol{I}$
x
$\boldsymbol{W}_1,\ \boldsymbol{b}_1$
$\boldsymbol{A}_1,\ \boldsymbol{c}_1$
$\boldsymbol{W}_2,\ \boldsymbol{b}_2$
$\boldsymbol{A}_2,\ \boldsymbol{c}_2$
$\boldsymbol{W}_3,\ \boldsymbol{b}_3$
$\boldsymbol{A}_3,\ \boldsymbol{c}_3$
$h(x) = h_3 \circ (\boldsymbol{I} + \boldsymbol{h}_2) \circ \boldsymbol{h}_1(x)$
first shallow sub-network block $\boldsymbol{h}_1$
second shallow sub-network block $\boldsymbol{I} + \boldsymbol{h}_2$
third shallow sub-network block h_3

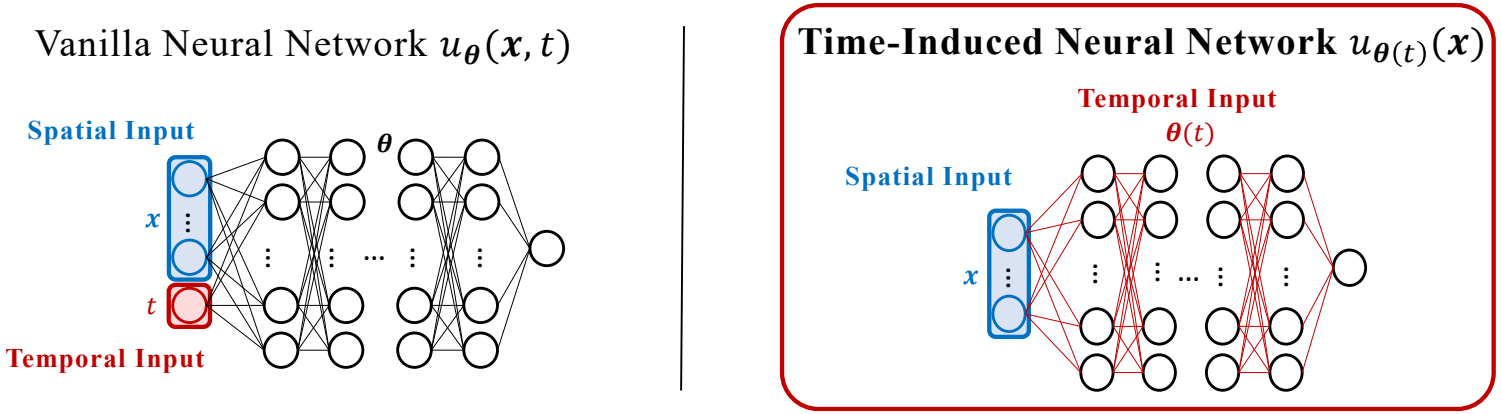

z
x
y


z
x
y

TINNs[3]

标准的时空 PINNs 将时间作为输入，但在所有时刻复用权重共享的单一网络，这就迫使同一组特征去表征差异显著的动力学行为．这种强耦合会降低精度，且在同时施加偏微分方程约束，边界约束与初始约束时，可能导致训练不稳定．

[3] 文提出了 TINNs 架构，它将网络权重参数化为时间的学习函数，使得有效的空间表征能够随时间演化，同时保留共享的结构．该文在多种含时偏微分方程上的实验表明：与 PINN 及其他强基线方法相比，TINN 的精度最高提升 4 倍，收敛速度最高加快 10 倍．



TINNs 的核心思想，是把 PINNs 的网络$u_\theta(\boldsymbol{x},t)$变为$u_{\theta(t)}(\boldsymbol{x})$，使得时间不进入网络输入，只进入参数空间.

考虑 L-layer TINNs 网络架构:

$u_\theta(\boldsymbol{x}) = \mathbf{W}_L\sigma(\mathbf{W}_{L-1}\sigma(...\sigma(\mathbf{W}_1\boldsymbol{x}+\mathbf{b}_1))+\mathbf{b}_{L-1})+\mathbf{b}_L$

其中$\boldsymbol{\theta}(t) = \{(\mathbf{W}_\ell,\mathbf{b}_\ell)\}_{\ell=1}^L$. $\mathbf{W}_\ell \in \mathbb{R}^{l_\ell\times l_{\ell-1}}, \mathbf{b}_\ell \in \mathbb{R}^{l_\ell}$,依赖 t , l_0为输入层(空间)维度.

总参数量为:

$$N_D = \sum_{\ell=1}^{L}(l_\ell l_{\ell-1}+l_\ell)$$

我们需要用 t 生成这N_D个参数.

一个自然的想法是使用 MLP 直接生成 $\boldsymbol{\theta}(t) \in \mathbb{R}^{N_D}$，但这样的方法会引入过多的参数.

另一个轻量的方法是令 $\boldsymbol{\theta}(t) = \boldsymbol{\omega} t + \boldsymbol{b}$.

此处是一次性生成所有层的参数，优点是能充分利用 gpu 的并行计算能力，缺点是优化器的自适应学习率,weight decay 无法针对每层参数进行调整.

如果设置第i层有四个独立的参数 $\mathbf{A}_{2i-1}, \mathbf{A}_{2i}, \mathbf{B}_{2i-1}, \mathbf{B}_{2i}$, 分层生成权重如下:

$$\mathbf{W}_i = t \cdot \mathbf{A}_{2i-1} + \mathbf{B}_{2i-1}$$
$$\mathbf{b}_i = t \cdot \mathbf{A}_{2i} + \mathbf{B}_{2i}$$

为了增强表达能力，论文中提出的方法是使用一个小神经网络输出 $\boldsymbol{\mathcal{N}} \in \mathbb{R}^{2L}$,然后使用一个门控机制得到：

$$\boldsymbol{\Phi}(t) = (\mathbf{1} - \boldsymbol{\alpha})t + \boldsymbol{\alpha} \odot \boldsymbol{\mathcal{N}}(t) \in \mathbb{R}^{2L}$$

$\boldsymbol{\alpha} \in \mathbb{R}^{2L}$,为可学习参数, $\boldsymbol{\Phi}(t) = (\Phi_1(t), \Phi_2(t), ..., \Phi_{2L}(t))^\top$.

$$\mathbf{W}_i = \Phi(t)_{2i-1} \cdot \mathbf{A}_{2i-1} + \mathbf{B}_{2i-1}$$
$$\mathbf{b}_i = \Phi(t)_{2i} \cdot \mathbf{A}_{2i} + \mathbf{B}_{2i}$$

TINNs[3]

能否推广 TINNs 的方法，在隐藏层嵌入空间坐标信息$(x_1, x_2, ..., x_d)$?

主要方向如下：

- 1 每层都嵌入所有物理坐标$(t, x_1, x_2, ..., x_d)$，

与 HyperPINN[4] 使用方程参数（如粘性系数）作为超网络的输入不同，用$(t, x_1, x_2, ..., x_d)$作为超网络的输入，让超网络生成 $\mathbf{\Phi}(t)$，然后把$\Phi_{i(t)}$经过仿射变换得到一个参数矩阵.

- 2 分层嵌入坐标信息，例如在第i层输入x_i，物理意义更明确.

预计该方法能够在具有复杂空间结构的偏微分方程上取得更好的效果.

Bibliography

[1] X. Feng, T. Tang, X. Wan, and T. Zhou, "Overcoming Spectral Bias via Cross-Attention." [Online]. Available: https://arxiv.org/abs/2512.18586

[2] S. Zhang, H. Zhao, Y. Zhong, and H. Zhou, "Fourier Multi-Component and Multi-Layer Neural Networks: Unlocking High-Frequency Potential." [Online]. Available: https://arxiv.org/abs/2502.18959

[3] C.-Y. Dai, C.-C. Chang, T.-S. Lin, M.-C. Lai, and C.-H. Lai, "TINNs: Time-Induced Neural Networks for Solving Time-Dependent PDEs." [Online]. Available: https://arxiv.org/abs/2601.20361

[4] F. de Avila Belbute-Peres, Y.-f. Chen, and F. Sha, "HyperPINN: Learning parameterized differential equations with physics-informed hypernetworks." [Online]. Available: https://arxiv.org/abs/2111.01008